

Databricks Free edition

Login to databricks free edition using the below given link:

https://login.databricks.com/?tuuid=5d7b8c2e-9025-400f-a4dc-4503a4f3a349&dbx_source=direct&rl_aid=eb104bb6-df53-478c-8490-19841b044eec

Session 4: Hands-on with databricks Notebooks

Steps to run a code in notebook:

Step: 1 Workspace → Home → Create → Folder

The screenshot displays the Databricks Free Edition interface. On the left, a sidebar contains navigation options: New, Workspace (highlighted), Recents, Catalog, Jobs & Pipelines, Compute, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie, Alerts, and Query History. The main workspace area shows a breadcrumb trail: Workspace > Home > Workspace. Below this, a table lists items in the workspace:

Name	Type	Owner	Created at
Bakehouse Sales Starter Space	Genie space	technicalsupport...	Aug 12, 2025, 02:...
Workspace Usage Dashboard	Dashboard	technicalsupport...	Aug 12, 2025, 02:...

A 'Create' dropdown menu is open, showing options: Folder (highlighted), Git folder, Notebook, File, Query, Dashboard, Genie space, Legacy Alert, Alert, and MLflow experiment. The background features a large watermark reading 'TRENDYTECH UPLIFT YOUR CAREER!'.

Once the folder is created,
Step 2: demo-folder → create → notebook

Workspace > Users > technicalsupport@trendytech.in >

demo-folder ☆

Send feedback Share Create

Search Type Owner Last modified

Name ↑ Type Owner

This folder is empty

- Folder
- Git folder
- Notebook**
- File
- Query
- Dashboard
- Genie space
- Legacy Alert
- Alert
- MLflow experiment

demo-notebook × +

File Edit View Run Help Python Tabs: ON ☆ Last edit was now Interrupt Connected Schedule

Just now (4s)

```
print(5+3)
```

8

Go to last run cell

Connected

Serverless

Environment version 3

More...

2 Python

Step 3: Create Compute:

Only serverless computer is present in free edition and classic compute is not present

Here's how it works in free edition:

- You get one small cluster automatically provisioned for you.
- You can attach/detach notebooks to this cluster.
- You cannot change its size, runtime, autoscaling, or configuration.
- It times out after some inactivity and you may need to restart it.

```
dbutils.fs.ls("/")
```

```
▶ ✓ 3 minutes ago (5s) 2
spark
<pyspark.sql.connect.session.SparkSession at 0x7f9c13ad9730>
```

```
▶ ✓ Just now (9s) 3
display(dbutils.fs.ls("/"))
> See performance \(2\)
```

	path	name	size	modificationTime
1	dbfs:/Volumes/	Volumes/	0	0
2	dbfs:/databricks-dataset...	databricks-dataset...	0	0

```
display(dbutils.fs.ls("/databricks-datasets/"))
```

```
▶ ✓ Just now (1s) 4 Python
display(dbutils.fs.ls("/databricks-datasets/"))
> See performance \(2\)
```

	path	name	size	modificationTime
1	dbfs:/databricks-datasets/COVID/	COVID/	0	1756376779645
2	dbfs:/databricks-datasets/README.md	README.md	976	1596557781000
3	dbfs:/databricks-datasets/Rdatasets/	Rdatasets/	0	1756376779645
4	dbfs:/databricks-datasets/SPARK_README.md	SPARK_README.md	3359	1596557823000
5	dbfs:/databricks-datasets/adult/	adult/	0	1756376779645
6	dbfs:/databricks-datasets/airlines/	airlines/	0	1756376779646
7	dbfs:/databricks-datasets/amazon/	amazon/	0	1756376779646
8	dbfs:/databricks-datasets/asa/	asa/	0	1756376779646
9	dbfs:/databricks-datasets/atlas_higgs/	atlas_higgs/	0	1756376779646
10	dbfs:/databricks-datasets/bikeSharing/	bikeSharing/	0	1756376779646
11	dbfs:/databricks-datasets/cctvVideos/	cctvVideos/	0	1756376779646

```
df = spark.read.option("header",
True).csv("/databricks-datasets/airlines/part-00000")
```

```
1 minute ago (3s) 5
df = spark.read.option("header", True).csv("/databricks-datasets/airlines/part-00000")
df: pyspark.sql.connect.dataframe.DataFrame = [Year: string, Month: string ... 29 more fields]
```

```
Just now (2s) 6
display(df)
> See performance (1) Optimize
```

	Year	Month	DayOfMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	C
1	1987	10	14	3	741	730	912	849
2	1987	10	15	4	729	730	903	849
3	1987	10	17	6	741	730	918	849
4	1987	10	18	7	729	730	847	849

To get a list of cancelled flights:

```
Just now (<1s) 7
filtered_df = df.filter("Cancelled == 1")
filtered_df: pyspark.sql.connect.dataframe.DataFrame = [Year: string, Month: string ... 29 more fields]
```

```
Just now (2s) 8
filtered_df.count()
> See performance (1) Optimize
5166
```

In Databricks Free Edition :

- DBFS root is disabled → you cannot write to /tmp, /dbfs, /mnt, etc.

Session 7: Managing Table Data and Metadata in Databricks Using Unity Catalog

In Databricks Free Edition, you do have a metastore, but it's:

- Shared across your workspace only (not Unity Catalog).
- All tables are managed tables (stored in Databricks' backend storage, not in DBFS).
- You cannot change the warehouse directory or inspect the actual storage location.

SQL code

how to check the current metastore

▶	✓	05:57 PM (57s)	2
%sql			
SELECT current_metastore();			
> See performance (1)			
Table ▾ +			
A ^B C current_metastore()			
1	aws:us-east-2:5f864d67-9999-41f0-8c1d-da085226b...		

SQL code:

▶	✓	06:01 PM (3s)	3
%sql			
SELECT current_catalog();			
> See performance (1)			
Table ▾ +			
A ^B C current_catalog()			
1	workspace		

Equivalent code using python:

```
spark.sql("SELECT current_catalog()").show()
spark.sql("SELECT current_schema()").show()
```

Sql code:

▶	✓	06:01 PM (3s)	4
%sql			
SELECT current_database();			
> See performance (1)			
Table ▾ +			
A ^B C current_database()			
1	default		

Python equivalent code :

```
print("Current Database:", spark.catalog.currentDatabase())
```

▶

▼

✓ Just now (6s)

5

```
%sql
SHOW CATALOGS;
```

>

[See performance \(1\)](#)

▶

📄

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [catalog: string]

Table ▼

+

	ABC catalog
1	..
2	samples
3	system
4	workspace

▶

▼

✓ Just now (4s)

```
%sql
SHOW SCHEMAS;
```

>

[See performance \(1\)](#)

▶

📄

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [databaseName: s

Table ▼

+

	ABC databaseName
1	default
2	information_schema

```
▶ ✓ Just now (2s) 7

%sql
USE CATALOG samples;

> See performance \(1\) Optimize
```

```
▶ ✓ Just now (5s) 8 SQL 🗑️ 🔍 🔗 ⋮

%sql
SHOW SCHEMAS;

> See performance \(1\) Optimize

_pyspark.sql.connect.dataframe.DataFrame = [databaseName: string]
```

Table	
databaseName	
1	bakehouse
2	information_schema
3	nyctaxi
4	tpch

Python equivalent code:

```
# Switch to catalog `samples`
spark.sql("USE CATALOG workspace")
```

```
spark.sql("show schemas")
```

The default catalog in free edition is 'workspace'

```
▶ ✓ Just now (2s) 9

%sql
USE CATALOG workspace;

> See performance \(1\)
```

```
▶ ✓ Just now (6s) 10

%sql
CREATE TABLE IF NOT EXISTS workspace.default.students (
  id INT,
  name STRING,
  score FLOAT
);

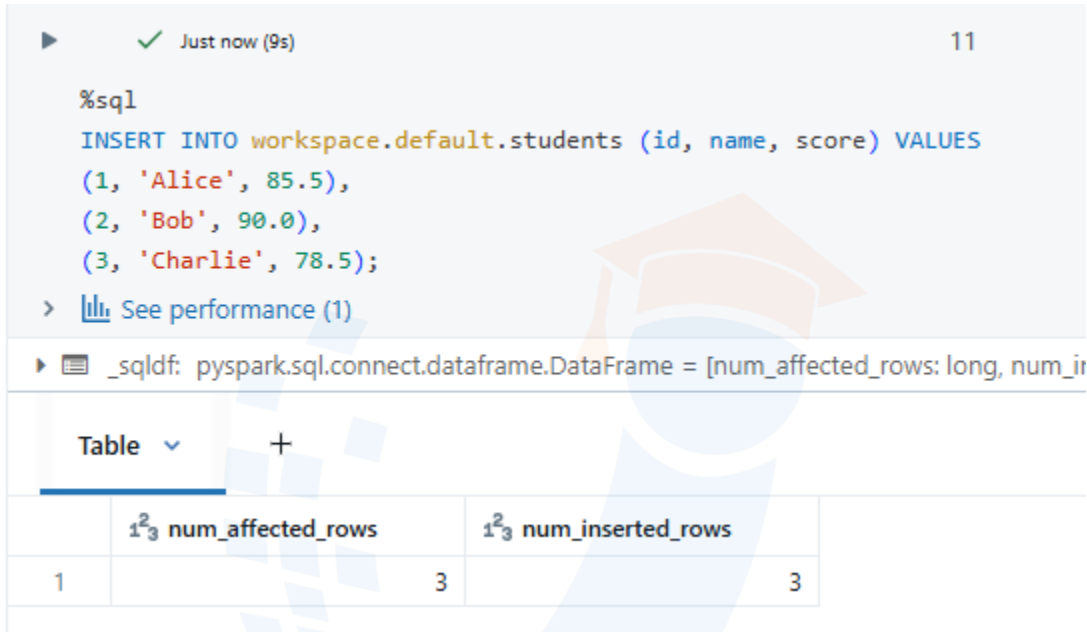
> See performance \(1\)
```

Equivalent python code:

```
spark.sql("""
CREATE TABLE IF NOT EXISTS workspace.default.students (
  id INT,
```

```
name STRING,  
score FLOAT  
)  
""")
```

SQL code:



The screenshot shows a SQL execution environment. At the top, a green checkmark and the text "Just now (9s)" indicate a successful execution. The SQL code executed is: `%sql
INSERT INTO workspace.default.students (id, name, score) VALUES
(1, 'Alice', 85.5),
(2, 'Bob', 90.0),
(3, 'Charlie', 78.5);`. Below the code, a link "See performance (1)" is visible. The execution result is displayed as a table with two columns: `num_affected_rows` and `num_inserted_rows`. The table shows that 3 rows were affected and 3 rows were inserted.

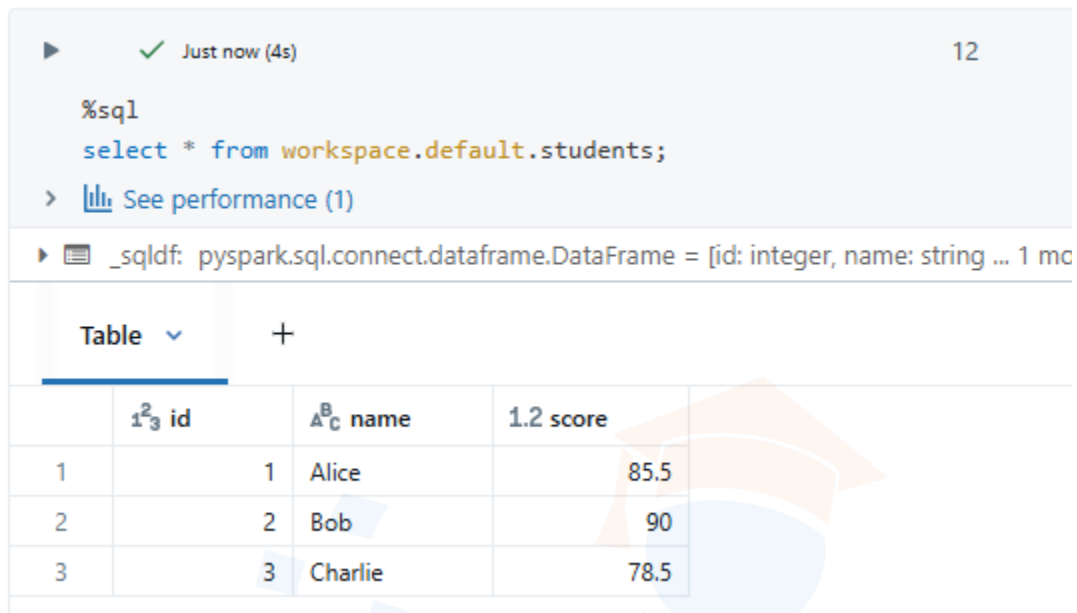
	<code>num_affected_rows</code>	<code>num_inserted_rows</code>
1	3	3

Equivalent Python Code:

```
spark.sql("""  
INSERT INTO workspace.default.students (id, name, score) VALUES  
(1, 'Alice', 85.5),  
(2, 'Bob', 90.0),  
(3, 'Charlie', 78.5)  
""")
```

TRENDY TECH
UPLIFT YOUR CAREER!

SQL code:



Just now (4s) 12

```
%sql
select * from workspace.default.students;
```

> [See performance \(1\)](#)

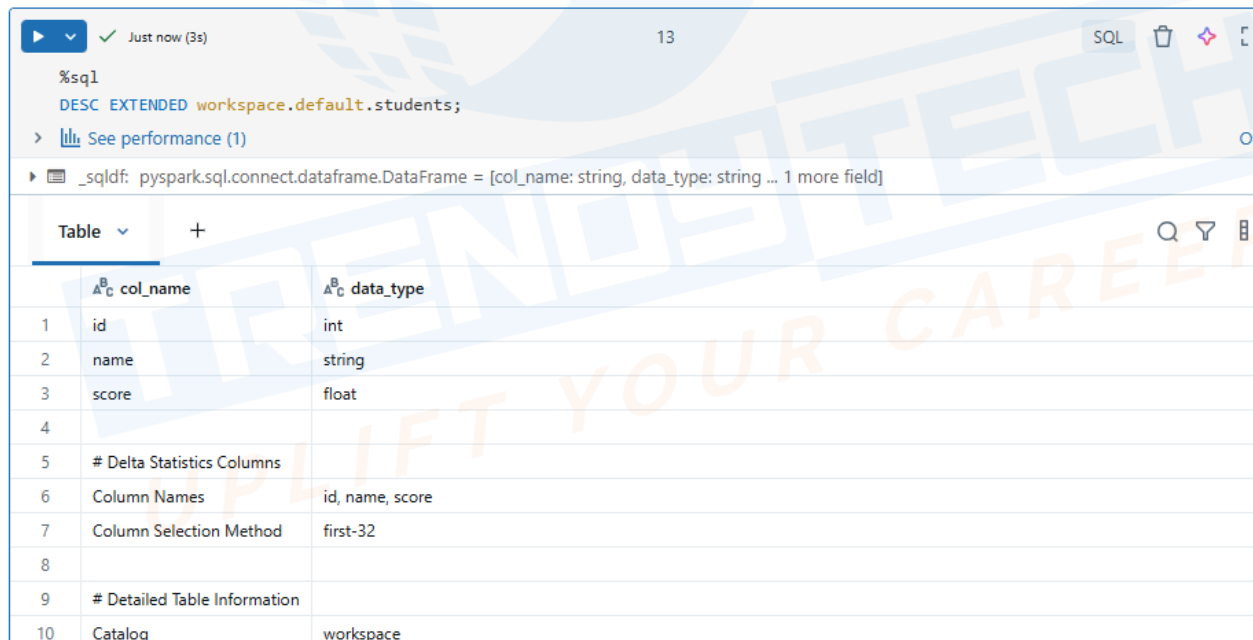
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [id: integer, name: string ... 1 mo

	id	name	score
1	1	Alice	85.5
2	2	Bob	90
3	3	Charlie	78.5

Python equivalent :

```
df = spark.sql("SELECT * FROM workspace.default.students")
df.show()
```

SQL code:



Just now (3s) 13 SQL

```
%sql
DESC EXTENDED workspace.default.students;
```

> [See performance \(1\)](#)

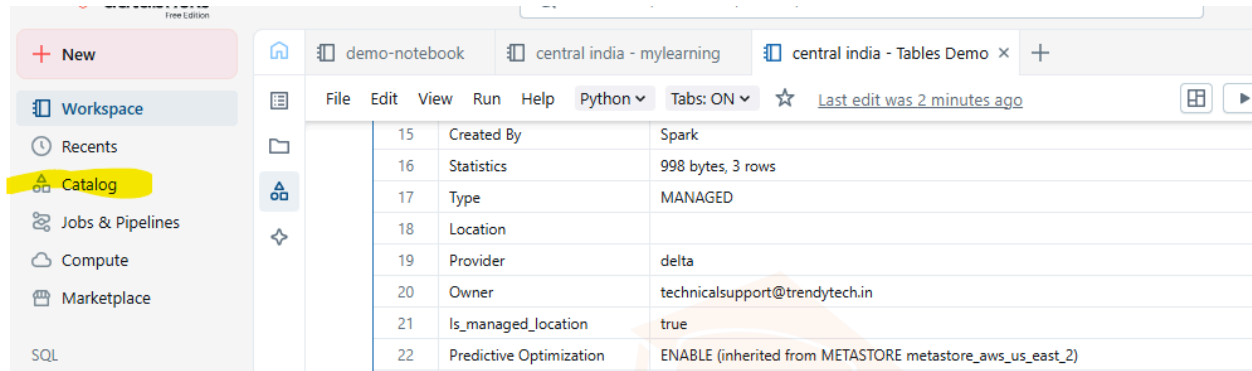
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [col_name: string, data_type: string ... 1 more field]

	col_name	data_type
1	id	int
2	name	string
3	score	float
4		
5	# Delta Statistics Columns	
6	Column Names	id, name, score
7	Column Selection Method	first-32
8		
9	# Detailed Table Information	
10	Catalog	workspace

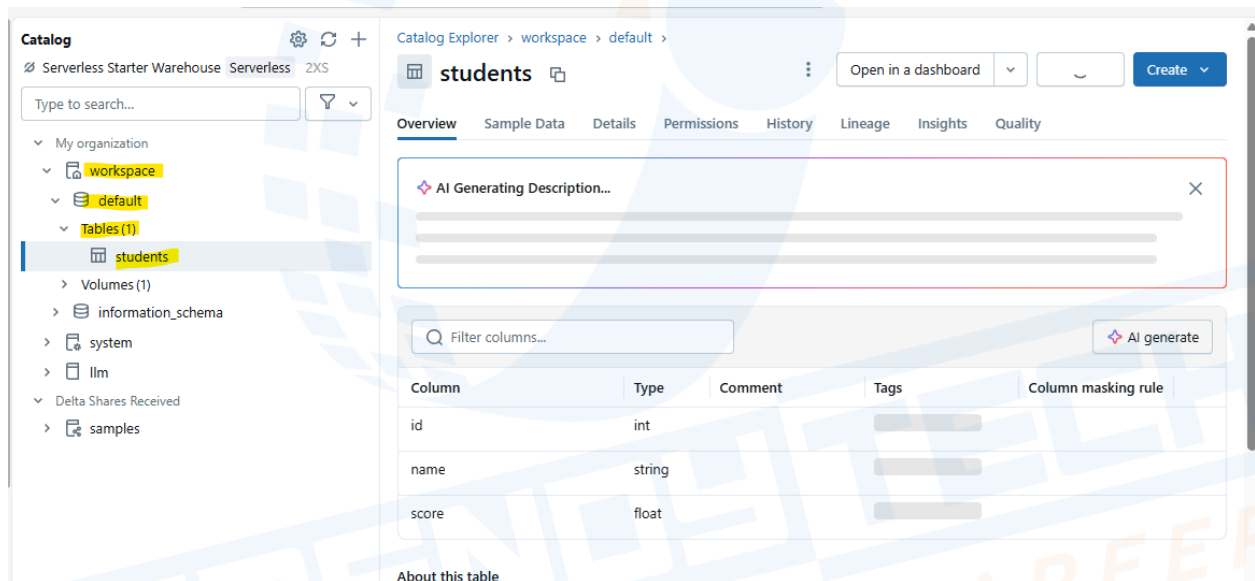
Python equivalent code:

```
df = spark.sql("DESC EXTENDED workspace.default.students")
df.show(truncate=False)
```

To see in catalog explorer:
Go to catalog



Catalog → Workspace → default → Tables → students



Hive_metastore is not available in the free edition

How to Use Azure Storage with Databricks Unity Catalog - need azure account again

%sql

```
create catalog my_catalog1
```

```
managed location
```

```
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog';
```

Python equivalent code:

```
spark.sql("""
```

```
CREATE CATALOG my_catalog1
```

MANAGED LOCATION

```
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog'
''')
```

```
%sql
use catalog my_catalog1;
```

Pyspark:

```
spark.sql("USE CATALOG my_catalog1")
```

Creating External Locations in Databricks Using SQL - need azure account again

```
%sql
SHOW EXTERNAL LOCATIONS;
```

Pyspark equivalent code:

```
df = spark.sql("SHOW EXTERNAL LOCATIONS")
df.show(truncate=False)
```

```
%sql
SHOW GRANTS ON EXTERNAL LOCATION adls_raw_storage_ext_loc;
```

Pyspark:

```
df = spark.sql("SHOW GRANTS ON EXTERNAL LOCATION adls_raw_storage_ext_loc")
df.show(truncate=False)
```

```
%sql
GRANT CREATE MANAGED STORAGE ON EXTERNAL LOCATION adls_raw_storage_ext_loc TO
`sumit.trendytech02_outlook.com#EXT#@sumittrendytech02outlook.onmicrosoft.com`;

SHOW GRANTS ON EXTERNAL LOCATION adls_raw_storage_ext_loc;
```

Pyspark:

```
# Grant permission
spark.sql("""
GRANT CREATE MANAGED STORAGE
ON EXTERNAL LOCATION adls_raw_storage_ext_loc
TO `sumit.trendytech02_outlook.com#EXT#@sumittrendytech02outlook.onmicrosoft.com`
""")
```

```
# Verify grants
```

```
df = spark.sql("SHOW GRANTS ON EXTERNAL LOCATION adls_raw_storage_ext_loc")
df.show(truncate=False)
```

```
%sql
CREATE CATALOG sales_catalog;
```

Pyspark:

```
spark.sql("CREATE CATALOG sales_catalog")
```

```
%sql
CREATE SCHEMA sales_catalog.retaildb;
```

Pyspark:

```
spark.sql("CREATE SCHEMA sales_catalog.retaildb")
```

```
%sql
CREATE TABLE sales_catalog.retaildb.sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
);
```

Pyspark:

```
spark.sql("""
CREATE TABLE sales_catalog.retaildb.sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
)
""")
```

```
%sql
INSERT INTO sales_catalog.retaildb.sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
```

```
(3, 'Charlie', 'Phone', 1, 500.00);

spark.sql("""
INSERT INTO sales_catalog.retaildb.sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00)
""")
```

Alternatively, you can also use the dataframe method:
from pyspark.sql import Row

```
# Create sample data
data = [
    Row(1, "Alice", "Laptop", 1, 1200.00),
    Row(2, "Bob", "Tablet", 2, 800.00),
    Row(3, "Charlie", "Phone", 1, 500.00)
]

# Create DataFrame
df = spark.createDataFrame(data, ["sale_id", "customer_name", "product", "quantity", "amount"])

# Append into table
df.write.mode("append").saveAsTable("sales_catalog.retaildb.sales")
```

```
%sql
DESCRIBE EXTENDED sales_catalog.retaildb.sales
```

Pyspark:

```
df = spark.sql("DESCRIBE EXTENDED sales_catalog.retaildb.sales")
df.show(truncate=False)
```

```
%sql
DROP CATALOG IF EXISTS sales_catalog CASCADE;
```

Pyspark:

```
spark.sql("DROP CATALOG IF EXISTS sales_catalog CASCADE")
```

```
%sql
CREATE EXTERNAL LOCATION adls_raw_storage_catalog_ext_loc URL
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog'
WITH (CREDENTIAL adls_raw_storage_cred)
```

```
COMMENT 'external location catalog level for raw sales data in adls';
```

Pyspark:

```
spark.sql("""
CREATE EXTERNAL LOCATION adls_raw_storage_catalog_ext_loc
URL 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location catalog level for raw sales data in adls'
""")
```

```
%sql
```

```
CREATE CATALOG sales_catalog
MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog';
```

Pyspark:

```
spark.sql("""
CREATE CATALOG IF NOT EXISTS sales_catalog
MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog'
""")
```

```
%sql
```

```
CREATE SCHEMA sales_catalog.retaildb;
```

Pyspark:

```
spark.sql("CREATE SCHEMA IF NOT EXISTS sales_catalog.retaildb")
```

```
%sql
```

```
CREATE TABLE sales_catalog.retaildb.sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
);
```

Pyspark:

```
spark.sql("""
CREATE TABLE IF NOT EXISTS sales_catalog.retaildb.sales (
    sale_id INT,
```

```
customer_name STRING,  
product STRING,  
quantity INT,  
amount DOUBLE  
)  
""")
```

```
%sql  
INSERT INTO sales_catalog.retaildb.sales VALUES  
(1, 'Alice', 'Laptop', 1, 1200.00),  
(2, 'Bob', 'Tablet', 2, 800.00),  
(3, 'Charlie', 'Phone', 1, 500.00);
```

```
Pyspark;  
spark.sql("""  
INSERT INTO sales_catalog.retaildb.sales VALUES  
(1, 'Alice', 'Laptop', 1, 1200.00),  
(2, 'Bob', 'Tablet', 2, 800.00),  
(3, 'Charlie', 'Phone', 1, 500.00)  
""")
```

Alternatively, using a DataFrame in PySpark:
from pyspark.sql import Row

```
data = [  
    Row(1, "Alice", "Laptop", 1, 1200.00),  
    Row(2, "Bob", "Tablet", 2, 800.00),  
    Row(3, "Charlie", "Phone", 1, 500.00)  
]
```

```
df = spark.createDataFrame(data, ["sale_id", "customer_name", "product", "quantity", "amount"])
```

```
df.write.mode("append").saveAsTable("sales_catalog.retaildb.sales")
```

```
%sql  
DESCRIBE EXTENDED sales_catalog.retaildb.sales
```

```
Pyspark:  
df = spark.sql("DESCRIBE EXTENDED sales_catalog.retaildb.sales")  
df.show(truncate=False)
```

```
%sql
CREATE EXTERNAL LOCATION adls_raw_storage_schema_ext_loc URL
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/schema'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location schema level for raw sales data in adls'
```

Pyspark:

```
spark.sql("""
CREATE EXTERNAL LOCATION adls_raw_storage_schema_ext_loc
URL 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/schema'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location schema level for raw sales data in adls'
""")
```

```
%sql
CREATE SCHEMA sales_catalog.retaildb1
MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/schema';
```

Pyspark:

```
spark.sql("""
CREATE SCHEMA IF NOT EXISTS sales_catalog.retaildb1
MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/schema'
""")
```

```
%sql
CREATE TABLE sales_catalog.retaildb1.sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
);
```

Pyspark:

```
spark.sql("""
CREATE TABLE IF NOT EXISTS sales_catalog.retaildb1.sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
```



```
    quantity INT,  
    amount DOUBLE  
)  
""")
```

```
%sql  
INSERT INTO sales_catalog.retaildb1.sales VALUES  
(1, 'Alice', 'Laptop', 1, 1200.00),  
(2, 'Bob', 'Tablet', 2, 800.00),  
(3, 'Charlie', 'Phone', 1, 500.00);
```

Pyspark:

```
spark.sql("""  
INSERT INTO sales_catalog.retaildb1.sales VALUES  
(1, 'Alice', 'Laptop', 1, 1200.00),  
(2, 'Bob', 'Tablet', 2, 800.00),  
(3, 'Charlie', 'Phone', 1, 500.00)  
""")
```

Alternate dataframe method:
from pyspark.sql import Row

```
data = [  
    Row(1, "Alice", "Laptop", 1, 1200.00),  
    Row(2, "Bob", "Tablet", 2, 800.00),  
    Row(3, "Charlie", "Phone", 1, 500.00)  
]
```

```
df = spark.createDataFrame(data, ["sale_id", "customer_name", "product", "quantity", "amount"])  
df.write.mode("append").saveAsTable("sales_catalog.retaildb1.sales")
```

Managed vs External Tables in Databricks - need azure account

```
%sql  
DROP TABLE sales_catalog.retaildb.sales;
```

Pyspark;

```
spark.sql("DROP TABLE IF EXISTS sales_catalog.retaildb.sales")
```

```
%sql  
SHOW TABLES DROPPED IN sales_catalog.retaildb;
```

Pyspark:

```
df = spark.sql("SHOW TABLES DROPPED IN sales_catalog.retaildb")  
df.show(truncate=False)
```

```
%sql  
select * from sales_catalog.retaildb.sales;
```

Pyspark:

```
df = spark.sql("SELECT * FROM sales_catalog.retaildb.sales")  
df.show(truncate=False)
```

```
%sql  
UNDROP TABLE sales_catalog.retaildb.sales;
```

Pyspark:

```
spark.sql("UNDROP TABLE sales_catalog.retaildb.sales")
```

```
%sql  
CREATE EXTERNAL LOCATION adls_storage_external_data_loc URL  
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external'  
WITH (CREDENTIAL adls_raw_storage_cred)  
COMMENT 'external location for handling external data in adls';
```

Pyspark:

```
spark.sql("""  
CREATE EXTERNAL LOCATION adls_storage_external_data_loc  
URL 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external'  
WITH (CREDENTIAL adls_raw_storage_cred)  
COMMENT 'external location for handling external data in adls'  
""")
```

```
%sql  
CREATE TABLE sales (  
    sale_id INT,  
    customer_name STRING,  
    product STRING,  
    quantity INT,  
    amount DOUBLE  
)  
location  
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external';
```

Pyspark:

```
spark.sql("""
CREATE TABLE sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
)
LOCATION 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external'
""")
```

```
%sql
desc extended sales;
```

Pyspark:

```
df = spark.sql("DESC EXTENDED sales")
df.show(truncate=False)
```

```
%sql
INSERT INTO sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00);
```

Pyspark:

```
spark.sql("""
INSERT INTO sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00)
""")
```

```
%sql
select * from sales;
```

Pyspark:

```
df = spark.sql("SELECT * FROM sales")
df.show(truncate=False)
```

```
%sql
DROP TABLE IF EXISTS sales;
```

Pyspark:

```
spark.sql("DROP TABLE IF EXISTS sales")
```

```
%sql
CREATE EXTERNAL LOCATION adls_storage_external_sales_data_loc URL
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external_1/sales
'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location for handling external sales data in adls'
```

Pyspark:

```
spark.sql("""
CREATE EXTERNAL LOCATION adls_storage_external_sales_data_loc
URL 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external_1/sales'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location for handling external sales data in adls'
""")
```

```
%sql
CREATE TABLE sales_ext (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
)
USING CSV
location
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external_1/sales
';
```

Pyspark:

```
spark.sql("""
CREATE TABLE IF NOT EXISTS sales_ext (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
)
USING CSV
LOCATION 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external_1/sales'
""")
```

```
%sql  
select * from sales_ext;
```

Pyspark:
df = spark.sql("SELECT * FROM sales_ext")
df.show(truncate=False)

```
%sql  
REFRESH TABLE training_centralus_dbws.default.sales_ext;
```

Pyspark:
spark.catalog.refreshTable("training_centralus_dbws.default.sales_ext")

